LAPORAN PRAKTIKUM
Struktur Data

MODUL 5
(Singly Linked List Bagian 2)



Disusun oleh:

Rezky Farrel
109082500203
S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
TAHUN 2026

Guided

1. **Guided 1**

   Source Code

```
MAIN.CPP
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;
int main() {
List L;
ElmList* hasil;
ElmList* P;
int total = 0;
CreateList(L);
// 1. Insert First
insertFirst(L, 2);
insertFirst(L, 0);
insertFirst(L, 8);
insertFirst(L, 12);
insertFirst(L, 9);
cout << "List awal: ";
printInfo(L);
// 2. Searching
hasil = findElm(L, 8);
if (hasil != nullptr) {
cout << "Angka 8 ditemukan." << endl;
} else {
cout << "Angka 8 tidak ditemukan." << endl;
}
// 3. Menghitung total nilai
P = L.First;
while (P != nullptr) {
total = total + P->info;
P = P->next;
}
cout << "Total nilai: " << total << endl;
// 4. Insert Last
insertLast(L, 20);
cout << "Setelah insertLast(20): ";
printInfo(L);
// 5. Insert After
insertAfter(L, 12, 99);
cout << "Setelah insertAfter(12, 99): ";
printInfo(L);
// 6. Update
update(L, 99, 100);cout << "Setelah update(99, 100): ";
printInfo(L);
// 7. Delete After
delAfter(L, 12);
cout << "Setelah delAfter(12): ";
printInfo(L);
// 8. Delete berdasarkan nilai
```

```
delP(L, 8);
cout << "Setelah delP(8): ";
printInfo(L);
// 9. Delete First
delFirst(L);
cout << "Setelah delFirst: ";
printInfo(L);
// 10. Delete Last
delLast(L);
cout << "Setelah delLast: ";
printInfo(L);
// 11. Menghitung jumlah node
cout << "Jumlah node: " << nbList(L) << endl;
// 12. Menghapus semua node
delAll(L);
cout << "Setelah delAll: ";
printInfo(L);
return 0;
}
```

SingleLinkList.cpp

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;
// Membuat list kosong
void CreateList(List &L) {
L.First = nullptr;
}
// Membuat node baru
ElmList* alokasi(int nilai) {
ElmList* P = new ElmList;
P->info = nilai;
P->next = nullptr;
return P;
}
// Membebaskan memori node
void dealokasi(ElmList* &P) {
delete P;
P = nullptr;
}
// Menampilkan semua isi list
void printInfo(List L) {
ElmList* P = L.First;while (P != nullptr) {
cout << P->info << " ";
P = P->next;
}
cout << endl;
}
// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
ElmList* P = L.First;
while (P != nullptr) {
```

```
if (P->info == nilai) {
return P;
}
P = P->next;
}
return nullptr;
}
// Menambahkan node di awal
void insertFirst(List &L, int nilai) {
ElmList* P = alokasi(nilai);
P->next = L.First;
L.First = P;
}
// Menambahkan node di akhir
void insertLast(List &L, int nilai) {
ElmList* P = alokasi(nilai);
if (L.First == nullptr) {
L.First = P;
} else {
ElmList* Q = L.First;
while (Q->next != nullptr) {
Q = Q->next;
}
Q->next = P;
}
}
// Menambahkan node setelah nilai tertentu
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
ElmList* Q = findElm(L, nilaiSebelum);
if (Q == nullptr) {
cout << "Nilai sebelumnya tidak ditemukan." << endl;
} else {
ElmList* P = alokasi(nilaiBaru);
P->next = Q->next;
Q->next = P;
}
}
// Menghapus node pertama
void delFirst(List &L) {
if (L.First == nullptr) {cout << "List kosong." << endl;
} else {
ElmList* P = L.First;
L.First = L.First->next;
dealokasi(P);
}
}
// Menghapus node terakhir
void delLast(List &L) {
if (L.First == nullptr) {
cout << "List kosong." << endl;
} else if (L.First->next == nullptr) {
```

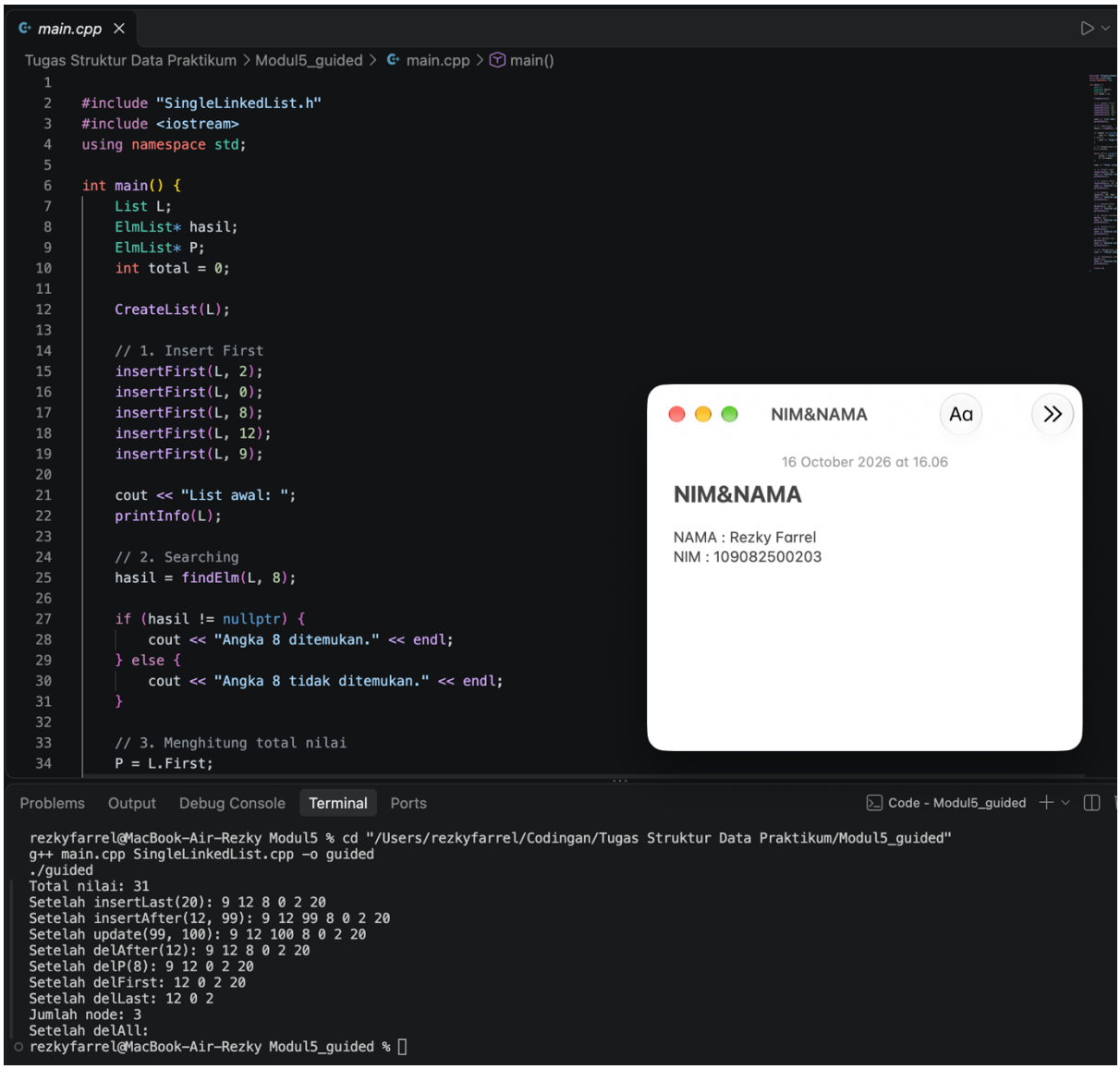
```
delFirst(L);
} else {
ElmList* P = L.First;
while (P->next->next != nullptr) {
P = P->next;
}
ElmList* Q = P->next;
P->next = nullptr;
dealokasi(Q);
}
}
// Menghapus node setelah nilai tertentu
void delAfter(List &L, int nilaiSebelum) {
ElmList* P = findElm(L, nilaiSebelum);
if (P == nullptr) {
cout << "Nilai sebelumnya tidak ditemukan." << endl;
} else if (P->next == nullptr) {
cout << "Tidak ada node setelah nilai tersebut." << endl;
} else {
ElmList* Q = P->next;
P->next = Q->next;
dealokasi(Q);
}
}
// Menghapus node berdasarkan nilai
void delP(List &L, int nilai) {
if (L.First == nullptr) {
cout << "List kosong." << endl;
} else if (L.First->info == nilai) {
delFirst(L);
} else {
ElmList* P = L.First;
while (P->next != nullptr &&
P->next->info != nilai) {
P = P->next;
}
if (P->next == nullptr) {
cout << "Nilai tidak ditemukan." << endl;
} else {
ElmList* Q = P->next;
P->next = Q->next;
dealokasi(Q);}
}
}
// Mengubah nilai data
void update(List &L, int nilaiLama, int nilaiBaru) {
ElmList* P = findElm(L, nilaiLama);
if (P == nullptr) {
cout << "Nilai tidak ditemukan." << endl;
} else {
P->info = nilaiBaru;
```

```
}
}
// Menghitung jumlah node
int nbList(List L) {
int jumlah = 0;
ElmList* P = L.First;
while (P != nullptr) {
jumlah++;
P = P->next;
}
return jumlah;
}
// Menghapus semua node
void delAll(List &L) {
while (L.First != nullptr) {
delFirst(L);
}
}
```

SingleLinkList.h

```
#ifndef SLL_H
#define SLL_H
struct ElmList {
int info;
ElmList* next;
};
struct List {
ElmList* First;
};
// Fungsi dasar
void CreateList(List &L);
ElmList* alokasi(int nilai); //alokasi untuk membuat node baru dan
mengembalikan alamat node tersebut
void dealokasi(ElmList* &P);
void printInfo(List L);
// Searching
ElmList* findElm(List L, int nilai);
// Insert
void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);
// Delete
void delFirst(List &L);
```

Screenshoot program



Deskripsi Program

Program ini menggunakan struktur data Single Linked List dalam bahasa C++ untuk menyimpan dan mengelola data berupa angka. Program terdiri dari tiga file, yaitu MAIN.CPP sebagai program utama, SingleLinkList.cpp yang berisi implementasi fungsi, dan SingleLinkedList.h yang berisi deklarasi struktur data serta fungsi yang digunakan.

Pada file SingleLinkedList.h, terdapat struktur ElmList yang menyimpan nilai (info) dan alamat node berikutnya (next), serta struktur List yang memiliki penunjuk ke node pertama (First). File SingleLinkList.cpp berisi fungsi-fungsi untuk membuat list kosong, membuat dan menghapus node, menampilkan data, mencari nilai, menambahkan node di awal atau akhir, menyisipkan node setelah nilai tertentu, mengubah nilai, menghapus node, menghitung jumlah node, dan menghapus seluruh isi list.

Pada file MAIN.CPP, program membuat list dan memasukkan lima angka menggunakan insertFirst, sehingga terbentuk urutan 9 → 12 → 8 → 0 → 2. Selanjutnya, program mencari

angka 8 dan menghitung total nilai, yaitu 31. Program kemudian menambahkan angka 20 di akhir, menyisipkan angka 99 setelah angka 12, lalu mengubahnya menjadi 100. Setelah itu, program menghapus angka 100, angka 8, node pertama, dan node terakhir secara berurutan. Hasil akhirnya adalah list 12 → 0 → 2 dengan jumlah 3 node. Terakhir, delAll menghapus seluruh node sehingga list kembali kosong.

LATIHAN SOAL MODUL

Link Repo Github Modul 5 : https://github.com/RezkyFarrel1/Struktur-Data/tree/main/Modul%205

1. **Soal 1**

   Source Code

Main.cpp

```cpp
#include <iostream>
#include "Singlylist.h"

using namespace std;

int main() {
    List L;
    address P1, P2, P3, P4, P5 = Nil;

    CreateList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    // Latihan 2: tampilkan isi list -> 9 12 8 0 2
    printInfo(L);

    // Latihan 3: cari elemen dengan info 8
    address hasil = findElm(L, 8);
    if (hasil != Nil) {
        cout << "8 ditemukan dalam list" << endl;
    } else {
        cout << "8 tidak ditemukan dalam list" << endl;
    }

    // Latihan 4: total info seluruh elemen
    cout << "Total info dari kelima elemen adalah " << sumInfo(L) << endl;

    return 0;
```

```
}
Singlylist.cpp

#include <iostream>
#include "Singlylist.h"

using namespace std;

void CreateList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void printInfo(List L) {
    address P = L.First;
    while (P != Nil) {
        cout << P->info << " ";
        P = P->next;
    }
    cout << endl;
}

void insertFirst(List &L, address P) {
    if (P != Nil) {
        P->next = L.First;
        L.First = P;
    }
}

address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil && P->info != x) {
        P = P->next;
    }
    return P;
}

int sumInfo(List L) {
    int total = 0;
    address P = L.First;
```

```
    while (P != Nil) {
        total += P->info;
        P = P->next;
    }
    return total;
}
```

Singlylist.h

```
#ifndef SINGLYLIST_H
#define SINGLYLIST_H
#include <iostream>

#define Nil NULL

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

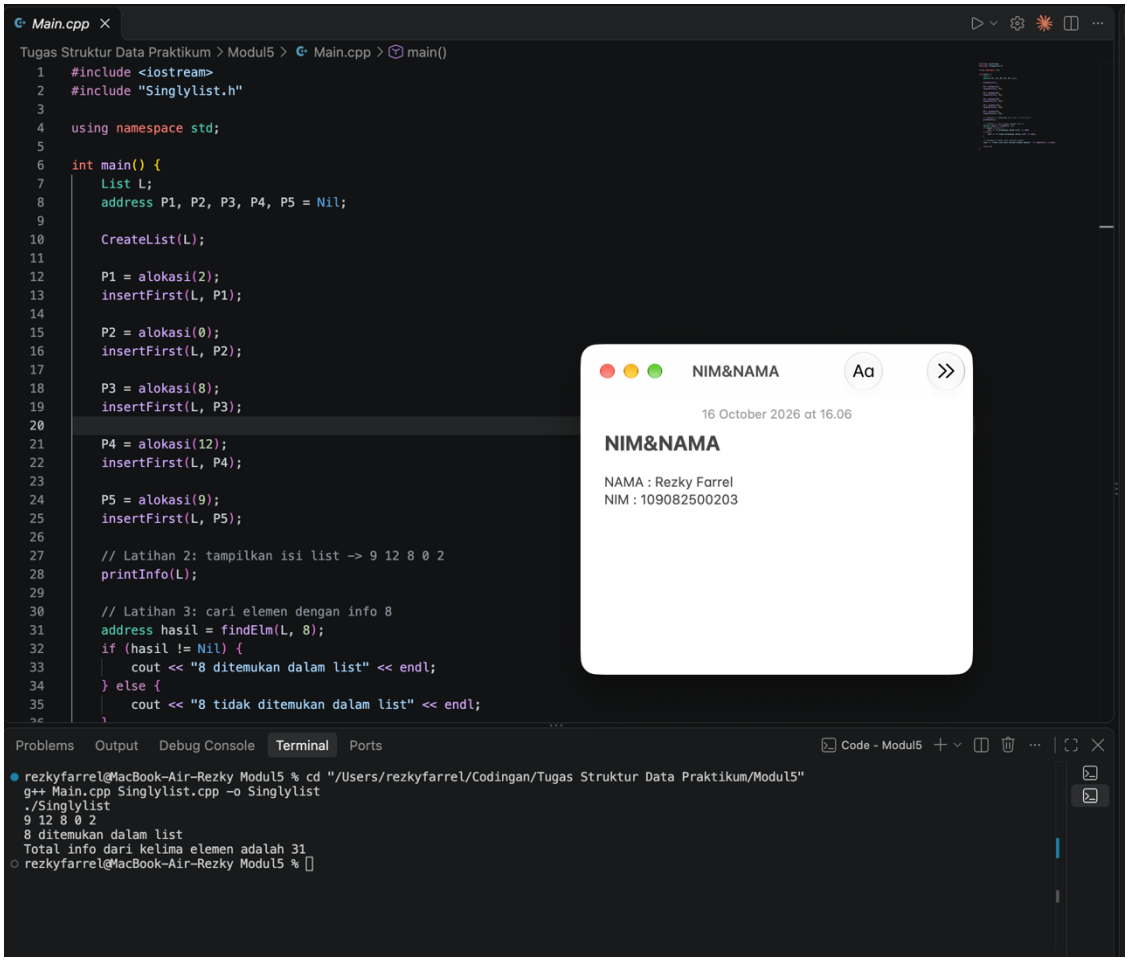
struct List {
    address First;
};

// Prototipe fungsi dasar
void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertFirst(List &L, address P);

// Prototipe fungsi tambahan untuk pencarian dan penjumlahan
address findElm(List L, infotype x);
int sumInfo(List L);

#endif
```

Screenshoot program



Deskripsi program

Program ini digunakan untuk membuat dan mengelola data menggunakan Single Linked List. Pertama, program membuat list kosong, kemudian memasukkan angka 2, 0, 8, 12, dan 9 ke bagian awal list. Karena setiap angka dimasukkan dari depan, urutan datanya menjadi 9, 12, 8, 0, dan 2. Setelah itu, program menampilkan isi list, mencari angka 8, dan menampilkan alamat memori tempat angka tersebut disimpan. Terakhir, program menghitung seluruh nilai dalam list dan menghasilkan total 31. Dengan program ini, kita bisa memahami cara menambahkan data, mencari data, dan menghitung jumlah nilai dalam Single Linked List.